The Foundations of Architecture

The Role of the Architect

Software Architecture & Design

MOVING FROM "HOW" TO "WHY."

What is Software Architecture?

The set of decisions that are expensive to change.

The blueprint of the system's components and their relationships.

A way to manage complexity and technical debt.

Defining software architecture

It is hard to define “Software Architecture”. (Why?).

Software Architecture embodies a massive amount of scope and responsibility.

These boundaries continue to expand.

Why does it move so fast?

Architects must make decisions within a constant changing environment.

Architecture, like art, can only be understood in context.

Factors

Structure of the system.

Architecture Characteristics.

Architecture Decisions.

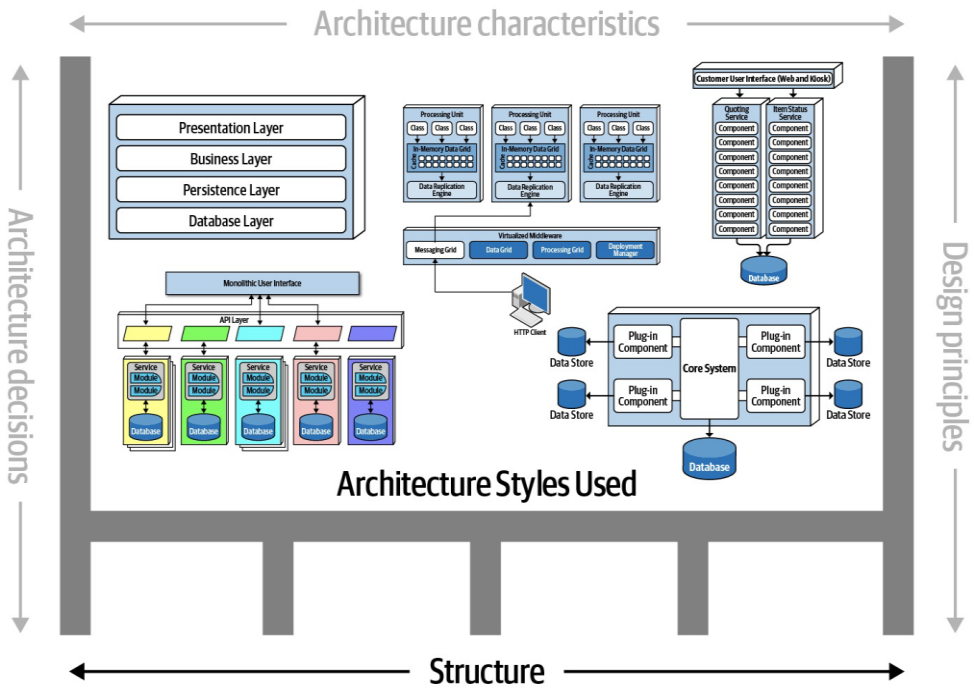
Design Principles.

Structure

Refers to the type of architectural styles.

E.g.: microservices, layered, microkernel.

Answers like: “It’s a microservices app” are incomplete.



Architecture Characteristics

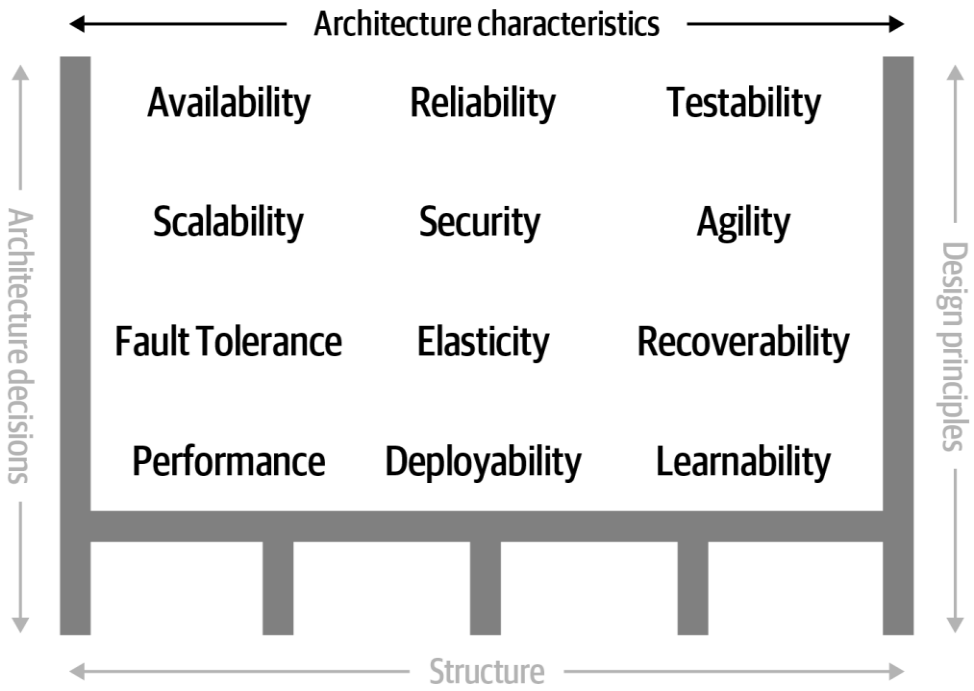
Define the success criteria of a system.

Characteristics do not require knowledge of functionality.

Also called “-ilities”.

The system must support the chosen ones.

It’s almost impossible to support all of them.



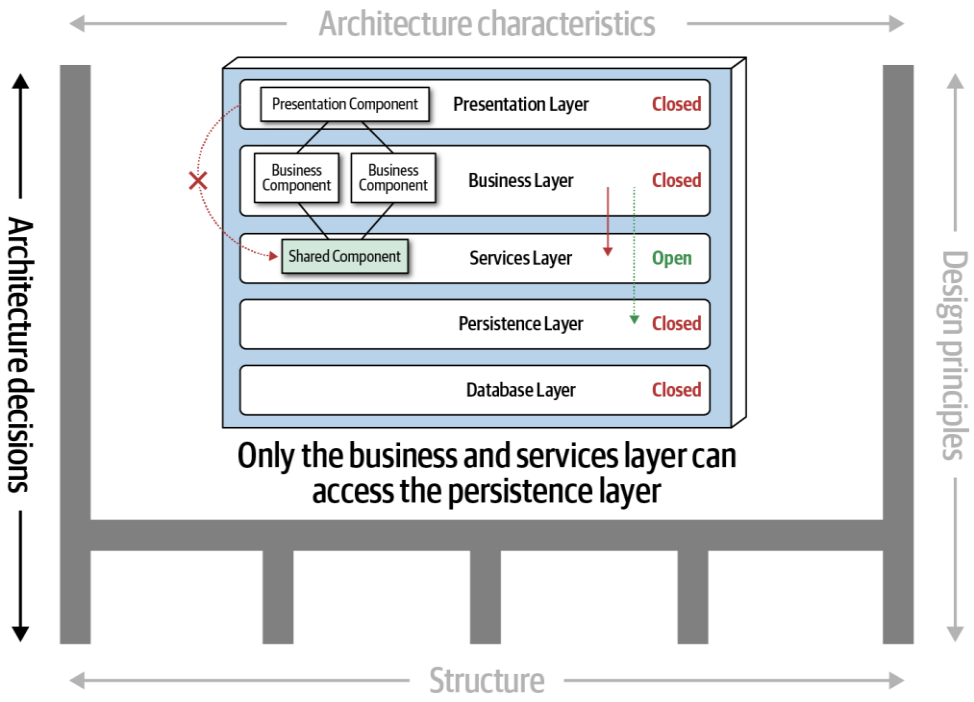
Architecture Decisions

Define the rules on how a system is built.

For instance: “only the business and service layers can access the database layer”.

Form the constraints of the system

Direct the development teams on what is and what is not allowed.

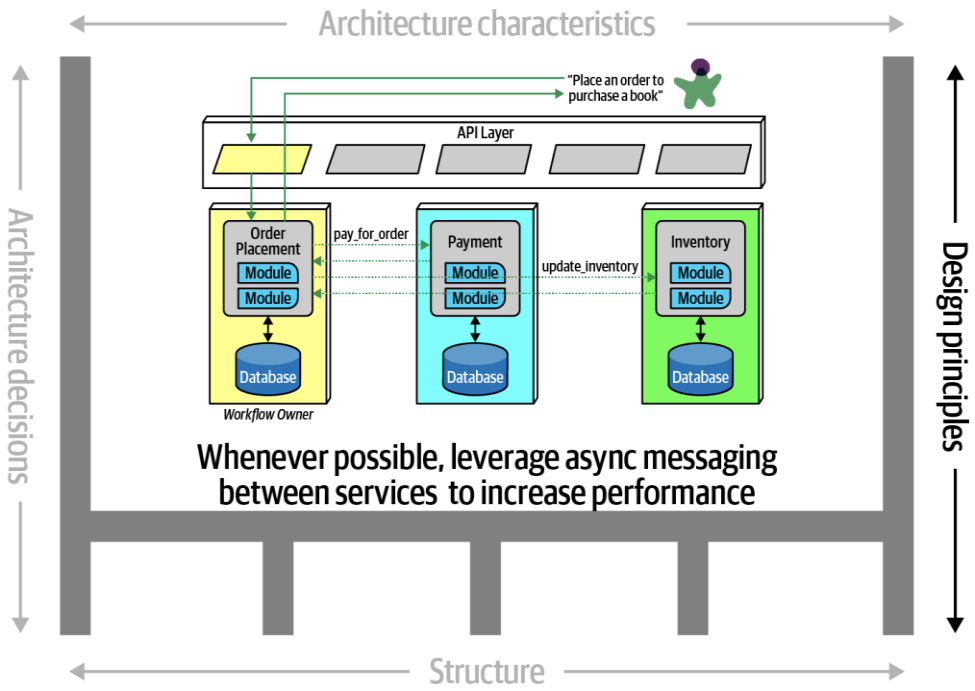


Design Principles

Differ from architecture decisions since design principles are guidelines.

For instance: synchronous vs asynchronous messages for communication.

Why is that this cannot be an architecture decision?



Software Design Specification (SDD/SDS)

Viewpoints

Viewpoints in SDD provide a holistic approach for stakeholders, ensuring that every aspect of the software's functionality and operation is considering through tailored. Think of it as the architectural blueprint detailing every corner of a building, enabling a diverse audience to visualize and understand the system from various angles.

Concerns

Addressing concerns proactively in SDD safeguards the software against future challenges. Factors like system performance, security, and scalability are thoroughly dissected to anticipate and neutralize potential issues, much like stress-testing architectural integrity against environmental uncertainties.

Views

The view methodically encapsulates different components of the software structure, much like isolating layers of a building to focus on plumbing, electrical, or ventilation systems. These views in SDD enable teams to concentrate on one segment at a time, facilitating a more focused and intricate development process.

Made with Gamma

Viewpoints

1 User Interface

Addresses the system's interaction with its users and focuses on usability and accessibility factors critical for user satisfaction.

2 System Integration

Concentrates on the system's compatibility and integrative abilities with other existing systems, highlighting interconnectivity.

3 Operational

Targets the conditions under which the software operates, streamlining processes for deployment, maintenance, and support.

4 Security

Zooms in on the protective mechanisms of the software, ensuring data integrity, confidentiality, and protection against breaches.

Made with Gamma

Concerns

1

Performance

Scrutinizing the efficiency of code execution and system responses to ensure fast and seamless operation.

2

Security

Fortifying the software against unauthorized access and vulnerabilities to uphold data protection standards.

3

Scalability

Planning for the system's growth and expansion, ensuring it performs effectively under varying load conditions.

Made with Gamma

Views

1

Logical View

Represents the system's functionality as perceived by end users and focuses on how features are initiated and delivered.

2

Process View

Examines the system’s runtime behavior and interactions, assessing processes, threads, and execution flows.

3

Development View

Details the system’s organization for development purposes, clarifying code structure, software layers, and programming environments.

Made with Gamma

Architectural Patterns

MVC

Disentangles user interface, business logic, and data model concerns, facilitating modular design and parallel development.

Layered

Establishes a hierarchy of responsibility, channeling requests through a vertical stack of independent layers for manageability and flexibility.

Microservices

Deconstructs monolithic structures into granular services, each with a focused purpose and capacity for independent scaling and evolution.

Made with Gamma

Package and Deployment Diagrams

Component	Purpose	Deployment Node
Web Server	Handles HTTP requests	Cloud Infrastructure
Application Logic	Processes business rules	Application Server
Database System	Manages data persistence	Database Server

Package and deployment diagrams serve as visual tools for developers to comprehend software distribution across various servers and environments. By codifying the abstract concept of deployment into tangible elements, architects can translate a software model into a viable, deployable product. It's a practice of bringing cohesion to the often-chaotic nature of system components' interactions, establishing visibility, and reinforcing structure across the software's lifecycle.

Made with Gamma

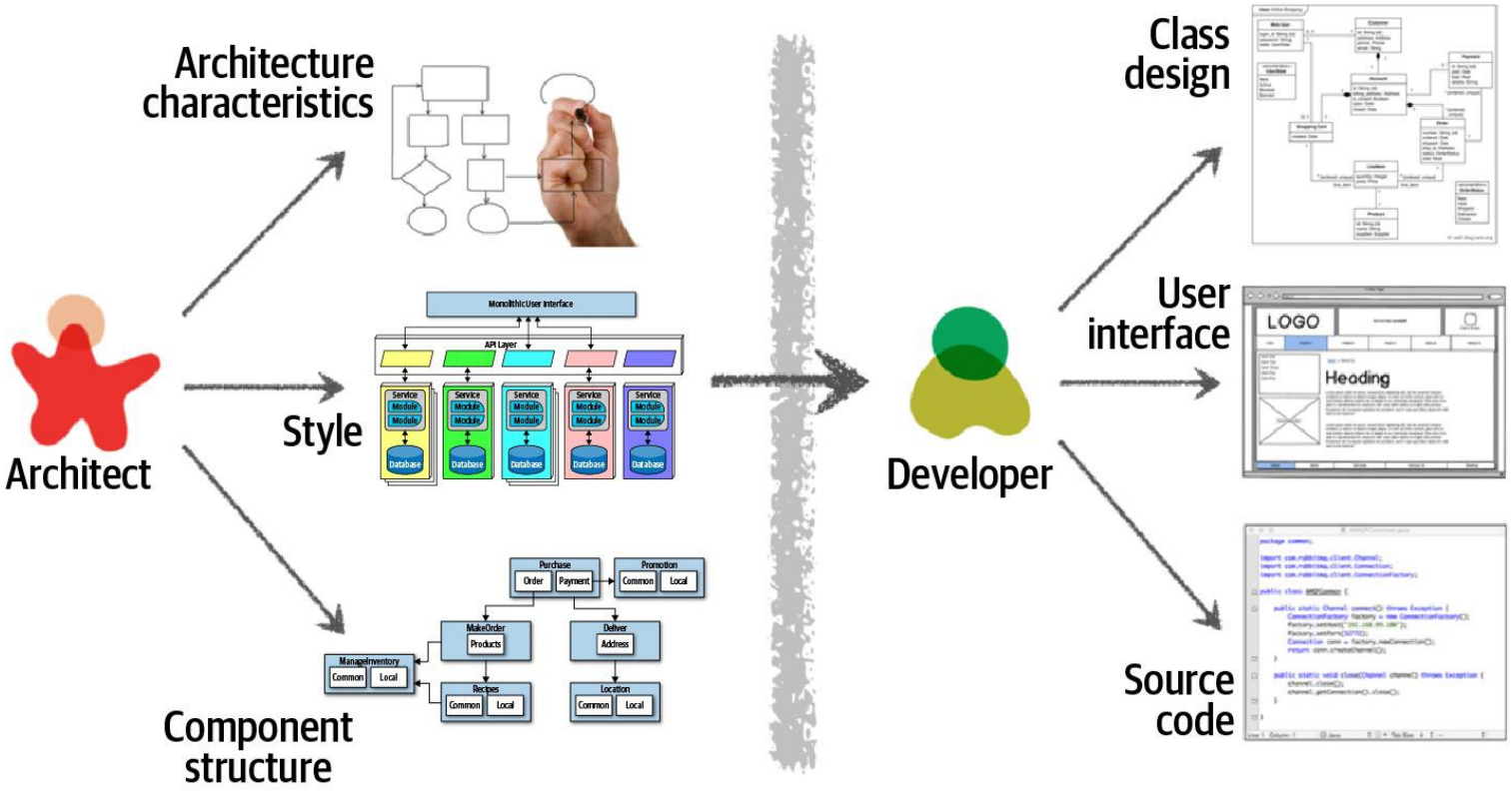
“Architecture is about the important stuff. Whatever that is.”

- RALPH JOHNSON

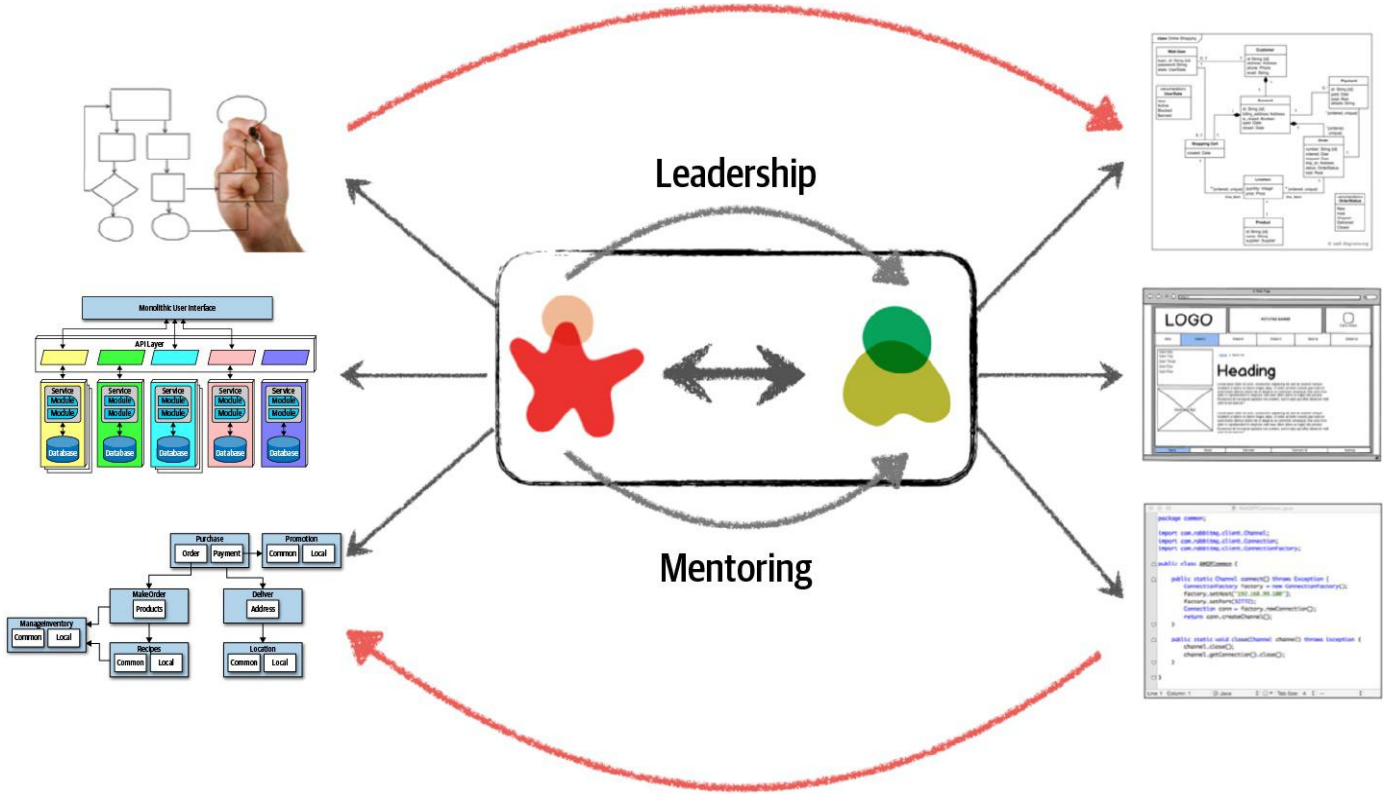
Architecture vs. Design

Feature	Architecture (Macro)	Design (Micro)
Focus	System Structure, Topology	Implementation, Logic
Impact	High (Hard to revert)	Low (Easier to refactor)
Decision	SQL vs. NoSQL	for loop vs map()

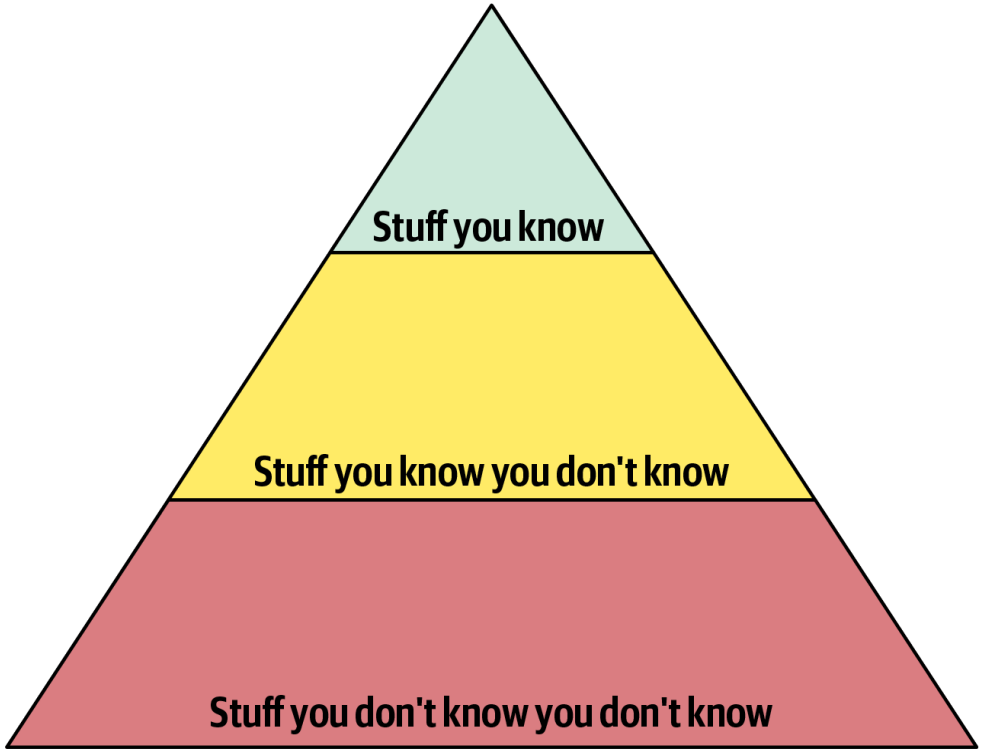
Architecture vs. Design

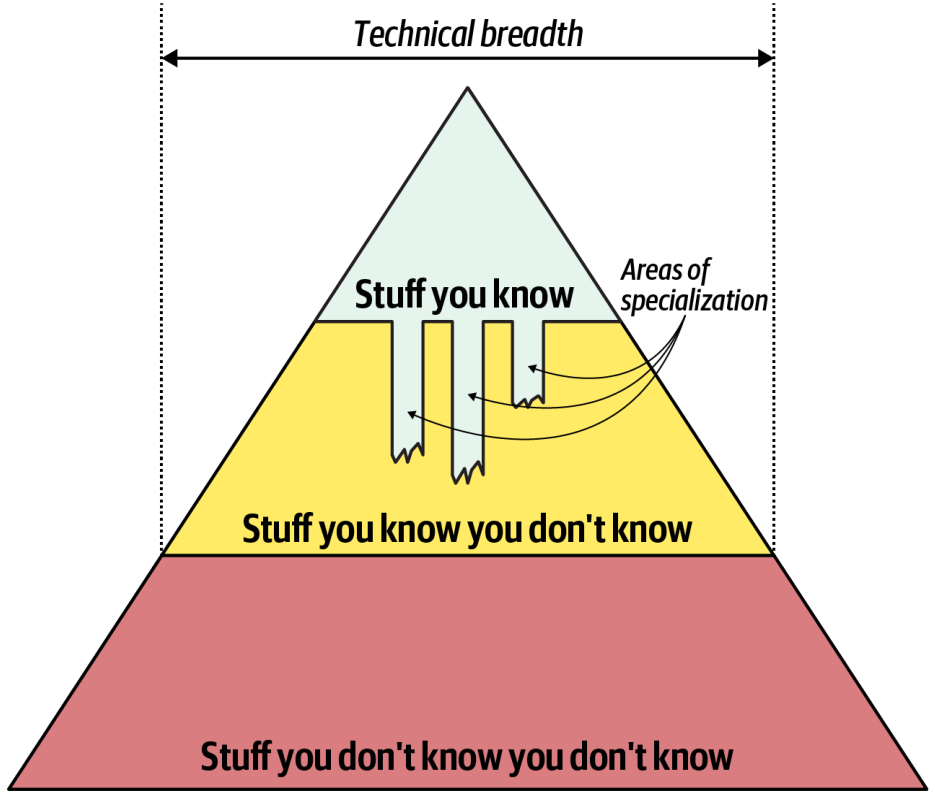


Architecture vs. Design



Technical Breadth

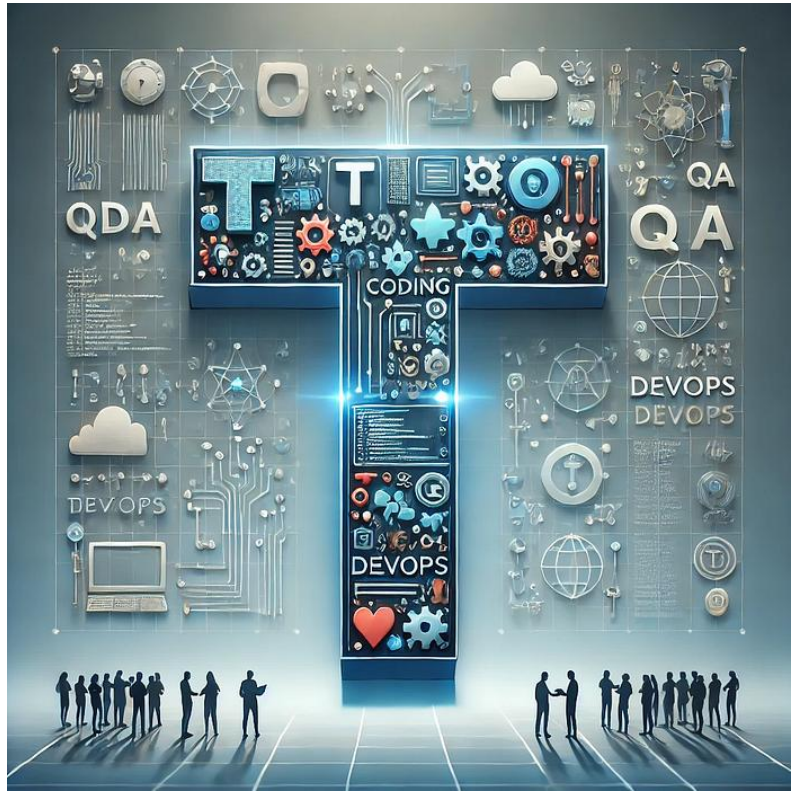




The T-Shaped Professional

Vertical Depth (Specialization): Represents deep knowledge, such as expertise in backend distributed systems, AI modeling, cybersecurity, coding, Debugging, Frameworks (e.g., Spring Boot, React).

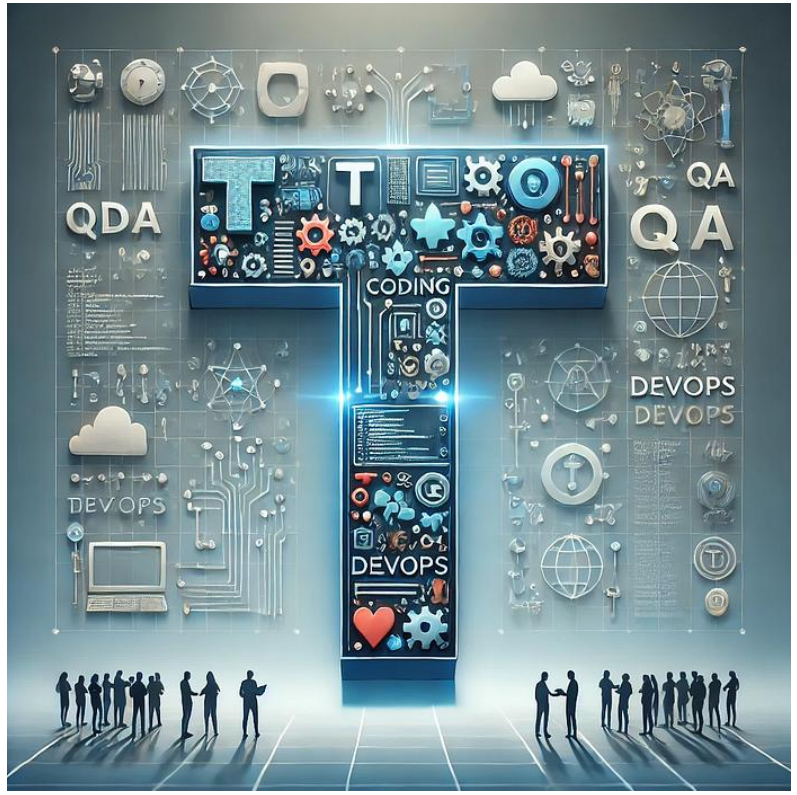
Horizontal Breadth (Generalization): Represents familiarity with related areas (e.g., DevOps, UI/UX, product management), enabling effective collaboration, Security, Cloud, Data Privacy, Business Strategy, Soft Skills.



The T-Shaped Professional

Benefits: These engineers bridge gaps, facilitate faster development, and improve team cohesion. They can also adapt to new technologies faster due to their foundational understanding.

Evolution: The traditional T-shape is shifting toward a wider horizontal bar and slightly shallower vertical bar due to the rise of AI tools, enabling developers to touch more areas of the tech stack.



Expectations of an Architect

What should a software architect do?

And most important, what they should not do?

Core expectations from a software architect:

- Make architecture decisions.
- Continually analyze the architecture.
- Keep current with the latest trends.
- Ensure compliance with decisions.
- Diverse exposure and experience.
- Possess interpersonal skills.
- Understand and navigate politics.

Expectations of an Architect

Make architectural decisions:

- Define the architectural decisions and design principles to guide the development.
- Keyword: guide. I.e., “Use a reactive-based framework for front end development”.
- Some specific technology may be used to preserve and architecture characteristic.

Continually analyze the architecture:

- Analyze the current architecture and the technology environment and make suggestions on improvement.
- Keep up the architecture vitality.

Keep current with the latest trends:

- Keep current with the new technologies and industry standards.

Expectations of an Architect

Ensure compliance with decisions:

- Imagine the previous data-layer access scenario. What happens if a developer access the database directly?

Diverse exposure and experience:

- An architect must be exposed to different types of technologies, frameworks, platforms, environments.
- Knowledge is power.
- Architects must stretch their comfort zone.

Have business domain knowledge:

- Architects must not only dominate technology but also business domains.
- Otherwise, it’s harder to understand the requirements and even the base problem.

Expectations of an Architect

Possess interpersonal skills:

- For instance: teamwork, facilitation and leadership.
- Lead development team's trough the implementation of the architecture.

Understand and navigate politics:

- Almost every decision will be challenged: product owners, project managers, business stakeholders.
- Also maybe challenged by development.
- Architects must justify and fight for their chosen decisions.

Architecture Roles

Role Title	Focus	Key Responsibilities
Enterprise Architect	Company-wide IT strategy	Aligns IT infrastructure with long-term business goals; maps IT landscape.
Solution Architect	Specific project solutions	Designs technical solutions to meet specific business problems; acts as a liaison.
Software/Application Architect	Individual software design	Defines application structure, coding standards, and tools.
Data Architect	Data management/storage	Defines policies for creating, storing, and retrieving data.
Cloud Architect	Cloud infrastructure	Designs, manages, and executes cloud computing strategies.
Security Architect	System security	Designs security structures to protect data and systems.
Technical/System Architect	Technical component design	Focuses on low-level designs, integrations, and technical expertise.

Useful skills

Engineering practices:

- Traditionally architecture was separated from development.
- It is good to separate software development process from engineering practices. What is the difference?
- The architect is often a tech lead, so they decide the engineering practices.

Operations/Dev ops: what are dev ops?

- New architectures consider that these operational concerns are better handled by operations.

Data:

- Database admins often work with architects to build data architecture for complex systems

Laws of Software Architecture

First law of Software Architecture:

- Everything in Software Architecture is a trade-off.
- Corollary 1:
  - If an architect thinks that they have discovered something that isn't a tradeoff, more likely they just haven't found the trade-off yet.

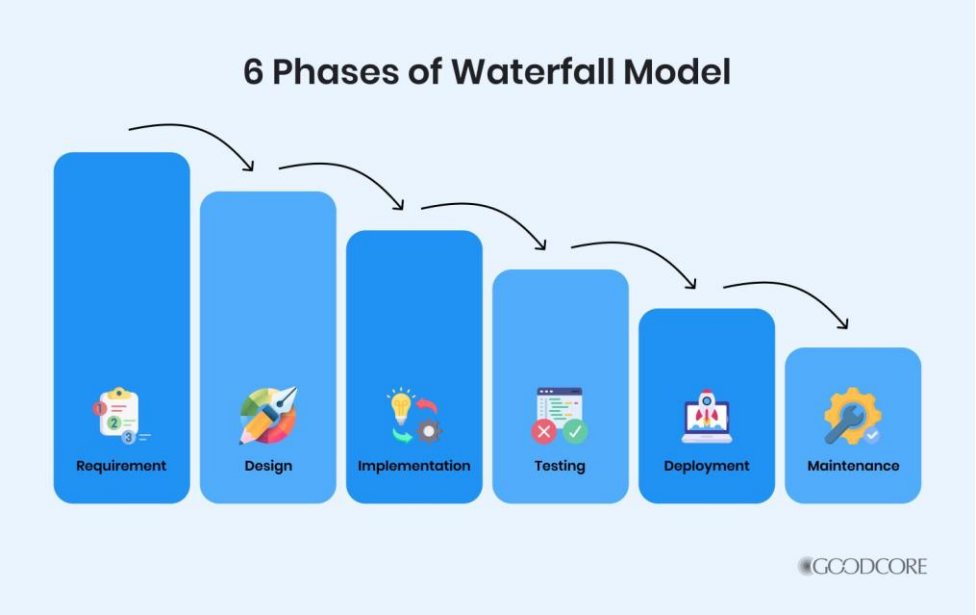
Second law of Software Architecture:

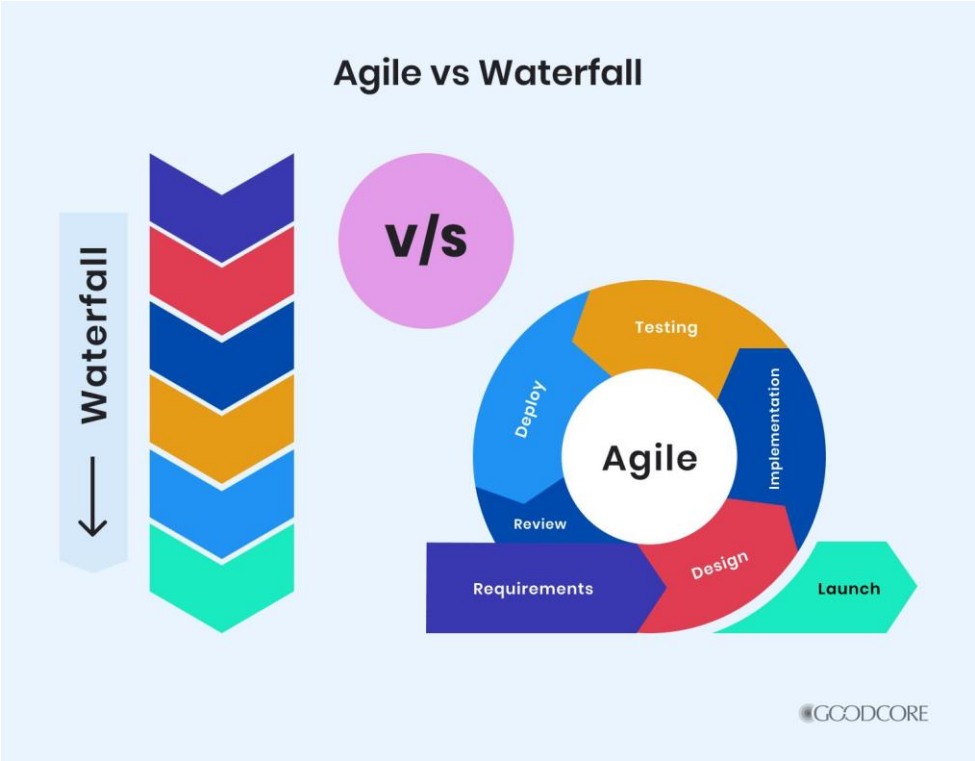
- Why is more important than how.*

An architect can look at an existing system and ascertain the structure, but they will struggle explaining why some choices were made.

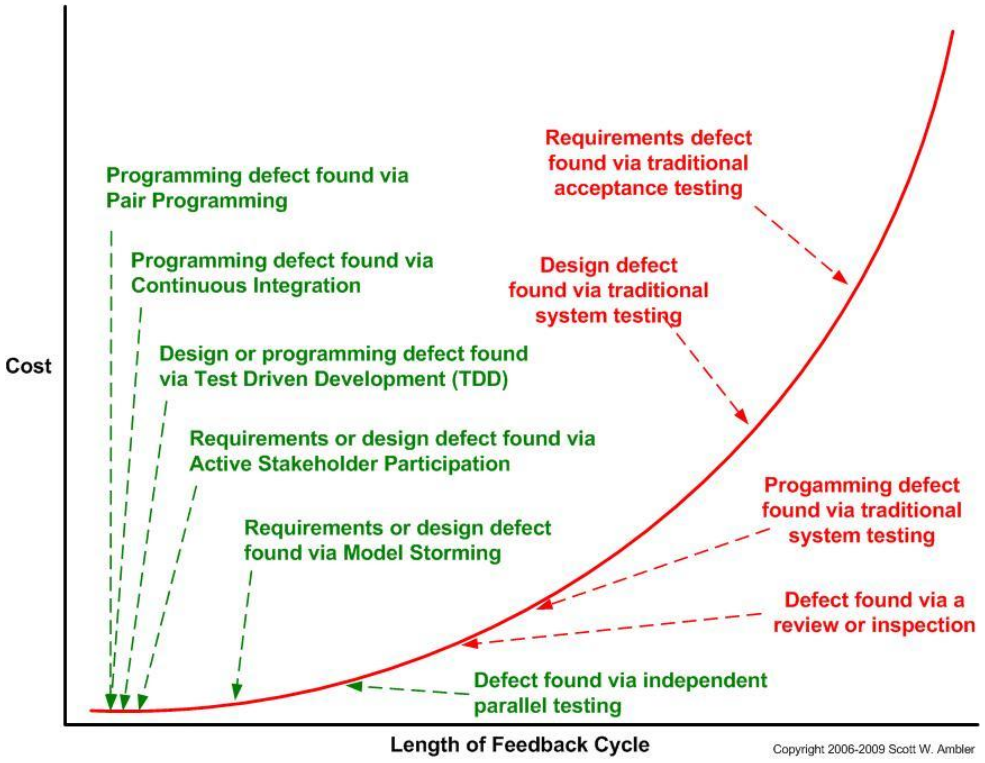
Architecture in the SDLC

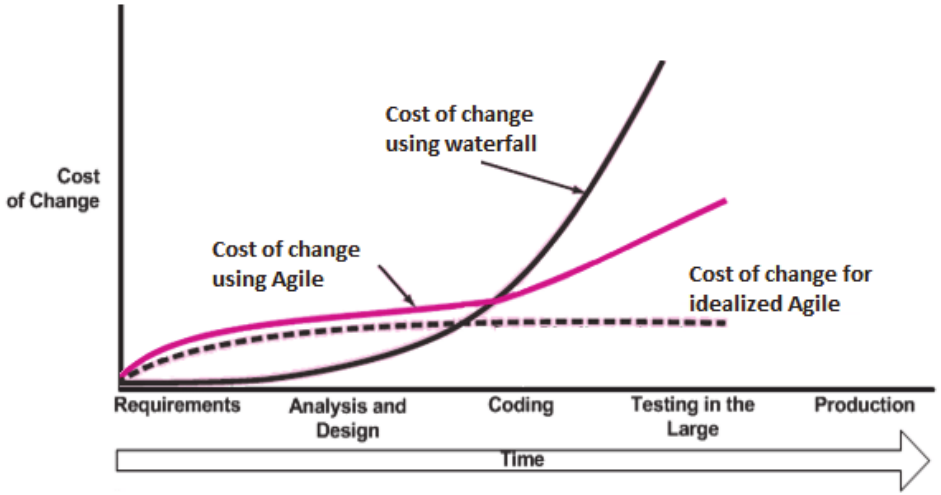
SDLC


6 Phases of Waterfall Model
Requirement
Design
Implementation
Testing
Deployment
Maintenance
GCODCORE


Agile vs Waterfall
Waterfall
v/s
Testing
Deploy
Agile
Implementation
Review
Requirements
Design
Launch
GCODCORE

The Cost of Change Curve





Architecture: Waterfall vs. Agile

Feature	Waterfall Architecture	Agile/DevOps Architecture
Timing	**BDUF:** Big Design Up Front.	**JEDI:** Just Enough Design Initially.
Documentation	Massive, comprehensive specs.	Lightweight ADRs & Living Diagrams.
Decision Making	Frozen before coding begins.	Deferred until the "Last Responsible Moment."
Risk Handling	High risk; flaws found at the end.	Lower risk; flaws found via early spikes.
Architect's Role	The "Police" (Enforces the plan).	The "Enabler" (Guides the team).

Key points

Topic		
The Waterfall Trap	“Walking on water and developing software from a specification are easy if both are frozen”	(Hint: They never are).
The Agile Myth	“Agile means we don't need architecture.”	**Correction:** Agile without architecture leads to a "Big Ball of Mud" where the system becomes impossible to change after 3 months.
The Sweet Spot	Modern architects use **Architectural Spikes** in Agile to research risky decisions without slowing down the sprint.	

Debunking BDUF

BDUF = (Big Design Up Front)

Architecture is not a 200-page document created before coding.

Agile Architecture = "Just Enough" design to start coding safely.

Evolutionary Architecture

Software evolves; the architecture must support “fitness functions”.

Design for decoupling so parts can be replaced without breaking the whole.

The Architect's Role in Scrum

Ceremony	Architect's Contribution
Sprint Planning	Identifies technical risks in user stories.
Refinement	Defines “Architectural Spikes” (Research tasks).
Review	Evaluates if the solution meets Quality Attributes.

Stakeholders: The Human Side of Architecture

MANAGING COMPETING INTERESTS AND TECHNICAL TRADE-OFFS

Who are the Stakeholders?

Users: Want features, ease of use, and speed.

Developers: Want maintainable code and modern tech.

Operations/DevOps: Want stability, easy deployments, and logs.

Security/Legal: Want data privacy (GDPR) and audit trails.

Management: Want low costs and fast time-to-market.

It's not just the person paying for the app.

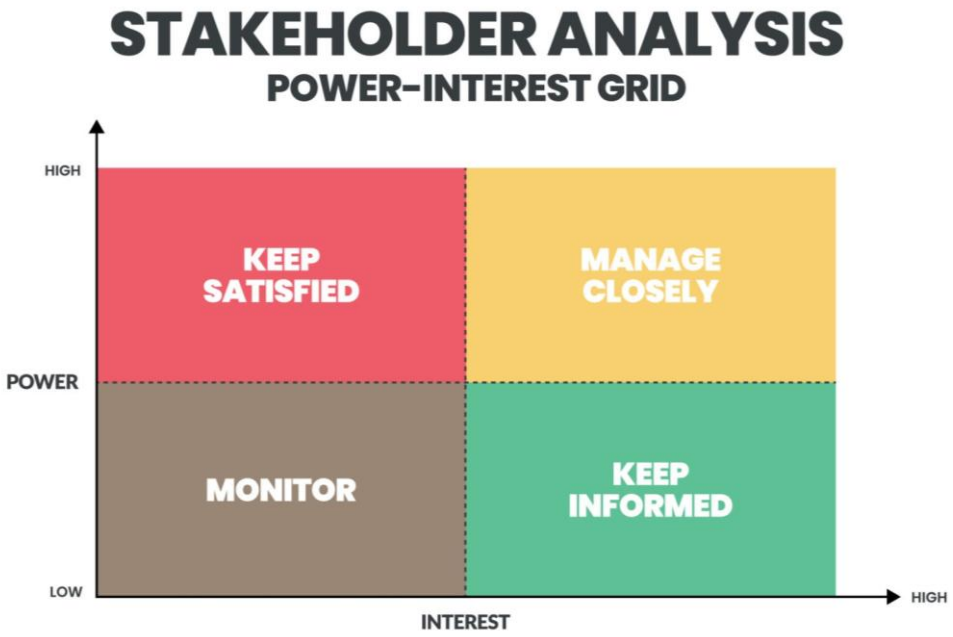
Mapping Influence (The Power/Interest Matrix)

High Power/High Interest: Manage Closely (Lead Architect, Product Owner).

High Power/Low Interest: Keep Satisfied (Legal, CFO).

Low Power/High Interest: Keep Informed (Developers, Users).

Low Power/Low Interest: Monitor (External vendors).



Architectural Concerns vs. Business Goals

Business Goal	Architectural Concern
“Release in 2 weeks”	Time-to-market vs. Tech Debt
“Handle 1M users”	Scalability & Elasticity
“Protect user data”	Security & Encryption
“Lower server costs”	Resource Efficiency/Serverless

The Architect as Negotiator

Architecture is a “Zero-Sum Game.”

You cannot maximize everything.

To gain **Scalability**, you might lose **Simplicity**.

To gain **Security**, you might lose **Performance**.

“It depends.”

- EVERY ARCHITECT IN THE WORLD

Activity: The Stakeholder Roleplay

Scenario: A startup is building a “Medical Records App”.

Teams of 5, roles: The Architect, The Doctor (User), The Lawyer (Legal), The Investor (Money), and the Developer.